

INDIVIDUAL PROJECT (LSDS)

WEEK 6 REPORT

(HBase Implementation – Car Price Analysis Project)

Name - Kuunal Mistry

AppID - 2410130

1) Objective of Week 6:

In Week 6, Apache HBase was implemented to store and retrieve car price data using a NoSQL column-oriented database model. Unlike Hive (which is SQL-based and used for batch analytics), HBase is designed for real-time data access and scalable distributed storage.

The objective of this week was to:

- Understand NoSQL database concepts
- Create HBase tables
- Design proper row keys
- Insert structured data
- Retrieve data using scan operation
- Compare Hive and HBase architecture

2) Services Used:

The following Hadoop ecosystem services were used:

- HDFS (Storage Layer)
- HBase (NoSQL Database)
- ZooKeeper (Coordination Service)
- YARN (Resource Management)

3) Table Creation in HBase:

First, the HBase shell was opened using:

hbase shell

A table named car_price_hbase was created with two column families:

create 'car_price_hbase', 'basic', 'specs'

Column Family Design:

- basic → Frequently accessed attributes (brand, price)
- specs → Technical attributes (engine, fuel type, year)

This separation improves data organization and scalability

```

 maria_dev@sandbox-hdp:~
 Using username "maria_dev".
 maria_dev@127.0.0.1's password:
Last login: Mon Mar  2 11:54:20 2026 from 172.18.0.3
[maria_dev@sandbox-hdp ~]$ hbase shell
HBase Shell; enter 'help<RETURN>' for list of supported commands.
Type "exit<RETURN>" to leave the HBase Shell
Version 1.1.2.2.6.5.0-292, r897822d4dd5956ca186974c10382e9094683fa29, Fri May 11 08:00:59 UTC
 2018

hbase(main):001:0> create 'car_price_hbase', 'details'
0 row(s) in 2.6190 seconds

=> Hbase::Table - car price hbase
```

```

hbase(main):012:0> disable 'car_price_hbase'
0 row(s) in 2.4920 seconds

hbase(main):013:0> drop 'car_price_hbase'
0 row(s) in 1.3200 seconds

hbase(main):014:0> create 'car_price_hbase', 'basic', 'specs'
0 row(s) in 2.2690 seconds

=> Hbase::Table - car_price_hbase
```

4) Row Key Design Strategy:

Instead of using simple numeric row keys (1, 2, 3), a structured row key format was designed:

Brand_Year_ID

Examples:

BMW_2018_1

Audi_2019_2

Why this is important:

- Enables faster lookup by brand and year
- Demonstrates real-world NoSQL modeling
- Avoids random row distribution issues

This reflects industry-level HBase schema design.

```
hbase(main):015:0> put 'car_price_hbase', 'BMW_2018_1', 'basic:brand', 'BMW'
0 row(s) in 0.0470 seconds

hbase(main):016:0> put 'car_price_hbase', 'BMW_2018_1', 'basic:price', '54157'
0 row(s) in 0.0090 seconds

hbase(main):017:0> put 'car_price_hbase', 'BMW_2018_1', 'specs:engine', '3.0'
0 row(s) in 0.0220 seconds

hbase(main):018:0> put 'car_price_hbase', 'BMW_2018_1', 'specs:fuel', 'Petrol'
0 row(s) in 0.0100 seconds

hbase(main):019:0> put 'car_price_hbase', 'BMW_2018_1', 'specs:year', '2018'
0 row(s) in 0.0310 seconds

hbase(main):020:0> put 'car_price_hbase', 'Audi_2019_2', 'basic:brand', 'Audi'
0 row(s) in 0.0190 seconds

hbase(main):021:0> put 'car_price_hbase', 'Audi_2019_2', 'basic:price', '51953'
0 row(s) in 0.0120 seconds

hbase(main):022:0> put 'car_price_hbase', 'Audi_2019_2', 'specs:engine', '2.0'
0 row(s) in 0.0120 seconds

hbase(main):023:0> put 'car_price_hbase', 'Audi_2019_2', 'specs:fuel', 'Diesel'
0 row(s) in 0.0230 seconds

hbase(main):024:0> put 'car_price_hbase', 'Audi_2019_2', 'specs:year', '2019'
0 row(s) in 0.0160 seconds
```

5) Data Insertion:

Data was inserted using the put command.

Example:

```
put 'car_price_hbase', 'BMW_2018_1', 'basic:brand', 'BMW'
```

6) Data Retrieval:

The data was retrieved using:

```
scan 'car_price_hbase'
```

The output displayed:

- Row Key
- Column Family
- Column Qualifier
- Timestamp
- Value

This demonstrates how HBase stores data in a column-oriented structure.

```
hbase(main):025:0> scan 'car_price_hbase'
ROW                                COLUMN+CELL
Audi_2019_2                        column=basic:brand, timestamp=1772542183795, value=Audi
Audi_2019_2                        column=basic:price, timestamp=1772542183834, value=51953
Audi_2019_2                        column=specs:engine, timestamp=1772542183873, value=2.0
Audi_2019_2                        column=specs:fuel, timestamp=1772542183908, value=Diesel
Audi_2019_2                        column=specs:year, timestamp=1772542185016, value=2019
BMW_2018_1                         column=basic:brand, timestamp=1772542174311, value=BMW
BMW_2018_1                         column=basic:price, timestamp=1772542174361, value=54157
BMW_2018_1                         column=specs:engine, timestamp=1772542174391, value=3.0
BMW_2018_1                         column=specs:fuel, timestamp=1772542174472, value=Petrol
BMW_2018_1                         column=specs:year, timestamp=1772542177134, value=2018
2 row(s) in 0.0760 seconds
```

7) Architectural Understanding:

HBase differs significantly from Hive.

Hive

- Structured SQL-based warehouse
- Used for batch analytics
- Suitable for aggregation queries (e.g., average price per brand)

HBase

- NoSQL column-oriented database
- Used for real-time lookup
- Suitable for fast row-level access
- Flexible schema

8) Hive vs HBase Comparison:

Feature	Hive	Hbase
Type	Data Warehouse	No SQL Database
Query Language	SQL	Shell commands/API
Schema	Fixed	Flexible
Processing	Batch	Real-time
Best Use case	Analytics	Instant Data Retrieval

9) Key Insights:

- HBase provides real-time row-level access, unlike Hive which is batch-based.
- Proper row key design is critical in NoSQL systems.
- Column families improve logical grouping of data.
- HBase is suitable for scalable, distributed applications such as e-commerce systems.
- Combining Hive and HBase provides both analytical and real-time capabilities.

10) Conclusion:

In Week 6, HBase was successfully implemented to store structured car price data using a NoSQL design approach. The project demonstrated:

- Schema design
- Column family separation
- Row key optimization
- Data insertion and retrieval
- Understanding of distributed NoSQL systems

This completes the NoSQL implementation phase of the Hadoop ecosystem project.